Requirement Generation Prompt

```
"""
You are an expert AI software engineering agent with strong capabilities in requirement
engineering, debugging, and code maintenance. Given an issue, you need to generate
requirements based on this issue.

-- Issue -- :
{problem_content}

Format your response as shown in <format_example>, and the format and content of the
requirement are also as follows. Include a THOUGHT section before your command where you
explain your reasoning process.

<format_example>
THOUGHT: Your reasoning and analysis here.
<requirement>
1. Background
- Main Functionality: Describe the main purpose or capabilities of the repository.
- Main Modules: Describe the different modules corresponding to functionalities and the
relationships between them.

2. Problem Overview
- Core Description: Provide a concise description of the problem.
- Problem Coverage: Indicate which functionalities and modules in the system are affected by
the faulty code.

3. Steps to Reproduce
- Preconditions: Specify the required state, data, or configuration before reproducing the
issue.
- Key Conditions: Clarify under what circumstances the error occurs, including specific
inputs, versions, etc..
- Reproduction Commands: Provide the complete procedure from start to triggering the issue.

4. Actual Behavior
- Erroneous Behavior: Describe the actual erroneous output or exception from the faulty
module.
- Correct Behavior: Describe the normal behavior of other modules using the affected code.

5. Expected Behavior
- Ideal Behavior: Describe the expected result when the system functions correctly.
- Success Criteria: Explain how to verify that the issue has been fixed.

6. Environment
- Dependencies and Imports: List the dependencies, required versions, APIs, libraries, or
modules necessary to reproduce and fix the issue.

7. Root Cause Analysis
- Error Cause: Infer the potential root cause of the issue from the observed symptoms.
- Code Paths: Point out the key functions, call chains, or logic paths where the problem
originates.

8. Solution
- Modification Location: Specify the files, functions, and code snippets that need to be
modified. Enclose the file path that needs to be modified within single parentheses.
- Modification Content: Provide a detailed description of the code snippets that need to be
added, deleted, or modified, and explain specifically how the modifications should be made.
- Impact Scope: Explain which modules will be affected and which correct functionalities
should remain unchanged.

9. Additional Notes
- Security, Compatibility, or Other Considerations: Provide additional notes that may affect
future maintenance, deployment, or security.
</requirement>
</format_example>

You need to complete all tasks within 50 steps and output the content in the above format.
"""
```

Patch Generation Prompt

````
"""
You will be given an issue resolution requirement and several related code files. Based on
the requirement and the provided code, you should write a patch to resolve the issue. The
patch must be applied only within the scope of the given files.
<requirement>
{issue_text}
</requirement>

<code_files>
{code_files}
</code_files>

Here is an example of a patch file. It consists of changes to the code base. It specifies
the file names, the line numbers of each change, and the removed and added lines. A single
patch file can contain changes to multiple files.
```patch
diff --git a/file.py b/file.py
--- a/file.py
+++ b/file.py
@@ -1,5 +1,5 @@
 def add(a, b):
-    return a + b
+    return a - b

 def mul(a, b):
     return a * b
```

* Please align the output precisely with the original code, and ensure the context lines
(starting with ' ') are an exact match for the target repository's code at the time of the
change.
* Ensure the file paths in the diff --git lines are relative to the repository root (e.g.,
django/forms/widgets.py) and use the a/ and b/ prefixes correctly.
* Please output your briefly chain-of-thought enclosed in <think>...</think>.
* Please output the patch in the ```patch ... ```.
"""
````

Test Generation Prompt

````
"""
You are an excellent test case researcher. Now, please write a Python test file based on a
repository-level code issue, requirements summarizing that issue and some related code.

The file should include multiple test cases: one part is reproduction tests, used to reproduce the
bug raised in the issue and check if the bug is correctly resolved after applying the patch; the
other part is regression tests, used to check if other originally correct functions still work
correctly after applying the patch. These two parts of test cases should be written in the same
file, using `assert` statements for judgment. The overall judgment logic has three cases: 1. If the
reproduction test successfully reproduces the bug and all regression tests pass, output "Reproduced
and Regression"; 2. If both the reproduction test and regression test pass, output "Resolved and
Regression"; 3. Otherwise, output "Error".

Below is a reproduction test example, which only includes reproduction tests, for your reference:
```python
import numpy as np
from astropy.io import fits
from astropy.io.fits import Column

def test_ascii_table_with_d_exponent():
    # Creating a FITS ASCII table with a column containing 'D' exponents
    data = np.rec.array([
        ('1.23D+01', '4.56D+02'),
        ('7.89D+03', '1.01D+04')
    ], dtype=[('col1', 'S10'), ('col2', 'S10')])

    col1 = Column(name='col1', format='A10', array=data['col1'])
    col2 = Column(name='col2', format='A10', array=data['col2'])

    hdu = fits.TableHDU.from_columns([col1, col2])

    try:
        # This will trigger the conversion of 'D' to 'E'
        converted_data = hdu.data
        # Attempt to convert the data to float
        try:
            col1_floats = converted_data['col1'].astype(float)
            col2_floats = converted_data['col2'].astype(float)
            assert np.allclose(col1_floats, [12.3, 7890.0]), "Test failed for col1"
            assert np.allclose(col2_floats, [456.0, 10100.0]), "Test failed for col2"
            print("Issue resolved")
        except ValueError as e:
            print("Issue reproduced")
            raise AssertionError("Conversion failed: 'D' exponents not handled correctly")
    except Exception as e:
        print("Other issues")
        raise AssertionError(e)

test_ascii_table_with_d_exponent()
```
The required information is as follows:
<issue>
{issue_text}
</issue>

<full_requirement>
{requirement_text}
</full_requirement>

<code_files>
{code_files}
</code_files>
Test file requirements are as follows:

* The generated test file may include both reproduction tests and regression tests.
* Multiple test functions can be generated, but it's best to strictly categorize them into three
cases.
* The test file must contain judgments for all three cases simultaneously, and must provide a
corresponding output print after execution.
* Please output the code in the ```python ... ```.
"""
````

Requirement Feedback Prompt

```
"""
I will provide you with an issue from a repository, a requirement summarized based on the issue and an incorrect patch with
test patch.
The format of the requirements document is as follows:
<requirement>
1. Background
- Main Functionality: Describe the main purpose or capabilities of the repository.
- Main Modules: Describe the different modules corresponding to functionalities and the relationships between them.

2. Problem Overview
- Core Description: Provide a concise description of the problem.
- Problem Coverage: Indicate which functionalities and modules in the system are affected by the faulty code.

3. Steps to Reproduce
- Preconditions: Specify the required state, data, or configuration before reproducing the issue.
- Key Conditions: Clarify under what circumstances the error occurs, including specific inputs, versions, etc..
- Reproduction Commands: Provide the complete procedure from start to triggering the issue.

4. Actual Behavior
- Erroneous Behavior: Describe the actual erroneous output or exception from the faulty module.
- Correct Behavior: Describe the normal behavior of other modules using the affected code.

5. Expected Behavior
- Ideal Behavior: Describe the expected result when the system functions correctly.
- Success Criteria: Explain how to verify that the issue has been fixed.

6. Environment
- Dependencies and Imports: List the dependencies, required versions, APIs, libraries, or modules necessary to reproduce and
fix the issue.

7. Root Cause Analysis
- Error Cause: Infer the potential root cause of the issue from the observed symptoms.
- Code Paths: Point out the key functions, call chains, or logic paths where the problem originates.

8. Solution
- Modification Location: Specify the files, functions, and code snippets that need to be modified.
- Modification Content: Provide a detailed description of the code snippets that need to be added, deleted, or modified, and
explain specifically how the modifications should be made.
- Impact Scope: Explain which modules will be affected and which correct functionalities should remain unchanged.

9. Additional Notes
- Security, Compatibility, or Other Considerations: Provide additional notes that may affect future maintenance, deployment, or
security.
</requirement>

Now, based on these information, please review the quality of the requirement by analyzing the incorrect patch from the
following three perspectives.

1. Conflict Reficiency(<Conflict>): The patch is incorrect because the requirement wrongly expresses the intent of the issue.
2. Omission Reficiency(<Omission>): The patch is incorrect because the requirement is missing part of the issue's intent.
3. Ambiguity Reficiency(<Ambiguity>): The requirement generally reflects the issue's intent correctly, but the patch is
incorrect because the requirements are not sufficiently detailed, which leads to misinterpretation.

Multiple of the above cases may exist simultaneously. Please first determine whether each of the three causes is present. Then,
in <Check></Check>, output the Conflict Reficiency items, the reasons for the errors, and suggestions for correction. Each item
must correspond exactly to one of the nine requirements listed above and use the same numbering. If you believe certain
requirement items are already correct, do not include them.

For Conflict Reficiency, emphasize how the requirement is incorrect and the direction for correction.
For Omission Reficiency, emphasize what content needs to be added.
For Ambiguity Reficiency, emphasize where the patch misinterpreted the requirement and how the requirement should be refined.

The output format should be as follows:

<Conflict>True/False</Conflict>
<Omission>True/False</Omission>
<Ambiguity>True/False</Ambiguity>
<Check>
1. Background: ...
3. Steps to Reproduce: ...
7. Root Cause Analysis: ...
</Check>

All information is provided below:

-- Issue --:
{problem_statement}

-- Requirement --:
{requirement_text}

-- Incorrect Patch --:
{patch_text}

-- Test Patch --:
{testcase}

"""
```

Requirement Refinement Prompt

```
"""
You are an expert AI software engineering agent with strong capabilities in requirement engineering, debugging, and
code maintenance. You are given a repository issue, a requirement generated from the issue, and an incorrect patch
produced based on that requirement. We have analyzed the incorrect patch and obtained findings about the problems
in the requirement. Based on these information, please reflect on the errors in the original requirement and
generate a new, corrected requirement.

The analysis is conducted from the following three perspectives.

1. Conflict Deficiency: The patch is incorrect because the requirement incorrectly expresses the intent of the
issue.
2. Omission Deficiency: The patch is incorrect because the requirement omits part of the issue's intent.
3. Ambiguity Deficiency: The requirement largely captures the intent of the issue, but the patch is incorrect
because the requirements are not described in sufficient detail, leading to misinterpretation.

The error type is reported in <Conflict></Conflict>, <Omission></Omission> and <Ambiguity></Ambiguity>, while
<Check></Check> contains the erroneous requirement items, the causes of the errors, and suggested corrections.

-- Issue -- :
{problem_content}

-- Previous Requirement -- :
{old_req}

-- Analysis -- :
{analysis}

Format your response as shown in <format_example>, and the format and content of the requirement are also as
follows. Include a THOUGHT section before your command where you explain your reasoning process.

<format_example>
THOUGHT: Your reasoning and analysis here.
<requirement>
1. Background
- Main Functionality: Describe the main purpose or capabilities of the repository.
- Main Modules: Describe the different modules corresponding to functionalities and the relationships between them.

2. Problem Overview
- Core Description: Provide a concise description of the problem.
- Problem Coverage: Indicate which functionalities and modules in the system are affected by the faulty code.

3. Steps to Reproduce
- Preconditions: Specify the required state, data, or configuration before reproducing the issue.
- Key Conditions: Clarify under what circumstances the error occurs, including specific inputs, versions, etc..
- Reproduction Commands: Provide the complete procedure from start to triggering the issue.

4. Actual Behavior
- Erroneous Behavior: Describe the actual erroneous output or exception from the faulty module.
- Correct Behavior: Describe the normal behavior of other modules using the affected code.

5. Expected Behavior
- Ideal Behavior: Describe the expected result when the system functions correctly.
- Success Criteria: Explain how to verify that the issue has been fixed.

6. Environment
- Dependencies and Imports: List the dependencies, required versions, APIs, libraries, or modules necessary to
reproduce and fix the issue.

7. Root Cause Analysis
- Error Cause: Infer the potential root cause of the issue from the observed symptoms.
- Code Paths: Point out the key functions, call chains, or logic paths where the problem originates.

8. Solution
- Modification Location: Specify the files, functions, and code snippets that need to be modified. Enclose the file
path that needs to be modified within single parentheses.
- Modification Content: Provide a detailed description of the code snippets that need to be added, deleted, or
modified, and explain specifically how the modifications should be made.
- Impact Scope: Explain which modules will be affected and which correct functionalities should remain unchanged.

9. Additional Notes
- Security, Compatibility, or Other Considerations: Provide additional notes that may affect future maintenance,
deployment, or security.
<>/requirement>
</format_example>

You need to complete all tasks within 50 steps and output the content in the above format.
"""
```

Test Refinement Prompt

````
"""
You are an excellent test case researcher. Now, please write a Python test file based on a
repository-level code issue, a requirements document summarizing that issue, and some related code.

The file should include multiple test cases: one part is reproduction tests, used to reproduce the
bug raised in the issue and check if the bug is correctly resolved after applying the patch; the
other part is regression tests, used to check if other originally correct functions still work
correctly after applying the patch. These two parts of test cases should be written in the same
file, using `assert` statements for judgment. The overall judgment logic has three cases: 1. If the
reproduction test successfully reproduces the bug and all regression tests pass, output \"Reproduced
and Regression\"; 2. If both the reproduction test and regression test pass, output \"Resolved and
Regression\"; 3. Otherwise, output \"Error\".

Below is a reproduction test example, which only includes reproduction tests, for your reference:
```python
import numpy as np
from astropy.io import fits
from astropy.io.fits import Column

def test_ascii_table_with_d_exponent():
    # Creating a FITS ASCII table with a column containing 'D' exponents
    data = np.rec.array([
        ('1.23D+01', '4.56D+02'),
        ('7.89D+03', '1.01D+04')
    ], dtype=[('col1', 'S10'), ('col2', 'S10')])

    col1 = Column(name='col1', format='A10', array=data['col1'])
    col2 = Column(name='col2', format='A10', array=data['col2'])

    hdu = fits.TableHDU.from_columns([col1, col2])

    try:
        # This will trigger the conversion of 'D' to 'E'
        converted_data = hdu.data
        # Attempt to convert the data to float
        try:
            col1_floats = converted_data['col1'].astype(float)
            col2_floats = converted_data['col2'].astype(float)
            assert np.allclose(col1_floats, [12.3, 7890.0]), \"Test failed for col1\"
            assert np.allclose(col2_floats, [456.0, 10100.0]), \"Test failed for col2\"
            print(\"Issue resolved\")
        except ValueError as e:
            print(\"Issue reproduced\")
            raise AssertionError(\"Conversion failed: 'D' exponents not handled correctly\")
    except Exception as e:
        print(\"Other issues\")
        raise AssertionError(e)

test_ascii_table_with_d_exponent()
```
The required information is as follows:
<issue>
{issue_text}
</issue>

<requirement>
{requirement_text}
</requirement>

<code_files>
{code_files}
</code_files>
Test file requirements are as follows:

* The generated test file may include both reproduction tests and regression tests.
* Multiple test functions can be generated, but it's best to strictly categorize them into three
cases.
* The test file must contain judgments for all three cases simultaneously, and must provide a
corresponding output print after execution.
* Please output your chain-of-thought enclosed in <think>...</think>.
* Please output the code in the ```python ... ```.
"""
````